

MMOClient

현재 코드 흐름 분석 + 기술면접 질문 은행

Unreal Engine 5.8 기반 C++ 클라이언트가 TCP 소켓, 전용 송수신 스레드, 게임 스레드 큐, Protobuf 패킷 디스패처를 통해 로그인·로비·방·이동 기능을 연결하는 흐름을 정리했다.

	이름
분석 기준	현재 작업 트리(미커밋 변경 포함), 정적 코드 분석
기준 커밋	21c7505 (2025-09-11, setting)
핵심 범위	Source/S1, Config, 생성된 Protocol/Struct/Enum 인터페이스
프로토콜 원본 참고	X:\Github\Server_std\Common\Protobuf\bin
작성일	2026-08-06

읽는 순서: ① 5분 요약 → ② 실제 호출 흐름 → ③ 구현 상태와 위험 → ④ 면접 답변 프레임 → ⑤ 질문 은행

1. 한눈에 보는 프로젝트

이 프로젝트의 중심은 UGameInstance를 애플리케이션 서비스 계층처럼 사용하고, TCP I/O는 FRunnable 기반 송수신 워커가 담당하며, 수신 패킷은 TQueue를 거쳐 게임 스레드에서 처리하는 구조다. UI/블루프린트는 UFUNCTION으로 요청을 보내고 Dynamic Multicast Delegate로 결과를 구독한다.

기능	클래스	기능	기능
부트/설정	DefaultEngine.ini, S1.cpp	LoginMap, BP_GameInstance, BP_GameMode 선택 및 모듈 로딩	설정됨
UI 접점	US1GameInstance UFUNCTION/Delegate	연결, 로그인, 방 목록·생성·입장·팀 변경 요청과 UI 알림	로비 중심 구현
도메인 상태	LobbyRooms, CurrentRoom, Players	서버 응답을 UE 친화적 USTRUCT/TArray/TMap으로 변환	방 상태 구현, 인게임 일부
패킷 계층	ServerPacketHandler	ID-함수 매핑, Protobuf 직렬화/역직렬화	모든 ID 등록, 다수 핸들러 비어 있음
세션/큐	PacketSession	워커 생명주기와 송수신 큐, 게임 스레드 디스패치	핵심 동작 구현
I/O 스레드	RecvWorker, SendWorker	TCP 정확 길이 수신/송신, 큐 전달	동작하나 종료·대기 개선 필요
플레이어	AS1Player, AS1MyPlayer	입력, 로컬 이동, 0.2초 이동 패킷, 원격 이동 기반	송신 구현, 원격 적용 주석 처리
직렬화	ProtobufCore + *.pb.*	스키마 기반 메시지와 생성 코드	연동됨

핵심 기술 스택

기술	기술	기술
엔진	Unreal Engine 5.8	S1.uproject EngineAssociation
언어/빌드	C++, UnrealBuildTool	S1.Build.cs, Runtime module
네트워크	TCP stream socket	FSocket, CreateSocket("Stream"), Connect
동시성	FRunnableThread + TQueue	RecvWorker/SendWorker, 두 공유 큐
프로토콜	4바이트 헤더 + Protobuf payload	uint16 size + uint16 id
입력	Enhanced Input	InputAction, MappingContext, EnhancedInputComponent
UI 연동	BlueprintCallable + Dynamic Multicast Delegate	GameInstance 공개 API와 이벤트

2. 객체 관계와 데이터 소유권

객체	객체	객체	객체
US1GameInstance	FSocket	raw pointer	연결 성공 후 보관, 해제는 소켓 서브시스템
US1GameInstance	PacketSession	TSharedPtr	송수신 워커와 큐의 수명 루트
PacketSession	RecvWorker/SendWorker	TSharedPtr	각 워커 객체를 보유
Worker	PacketSession	TWeakPtr	순환 참조 방지, Pin 성공 시 큐 접근
PacketSession	수신/송신 데이터	TQueue	네트워크 스레드와 게임 스레드의 경계
US1GameInstance	로비/방 상태	UPROPERTY 값 타입	블루프린트가 읽는 클라이언트 캐시

클래스	데이터	포인터	비고
US1GameInstance	플레이어 액터	raw UObject pointer/TMap	스폰·디스폰 관리 예정
AS1Player	MoveInfo 2개	new/delete raw pointer	현재 상태와 목적 상태; 값 멤버/스마트 포인터로 단순화 가능

스레드 경계의 핵심은 네트워크 워커가 UE 월드/액터를 직접 만지지 않고 큐에 바이트 배열만 넣는 것이다. 그 후 블루프린트 또는 게임 루프에서 US1GameInstance::HandleRecvPackets()를 호출해야 실제 핸들러가 게임 상태와 UI를 갱신한다.

3. 시작부터 종료까지 전체 실행 흐름

순서	이벤트	호출 함수	비고
1	엔진 부팅	DefaultEngine.ini	LoginMap, BP_GameInstance, BP_GameMode 로드
2	UI/블루프린트 요청	ConnectToGameServer()	127.0.0.1:7777 TCP 연결 시도
3	세션 생성	MakeShared	패킷 핸들러 테이블 초기화
4	워커 시작	PacketSession::Run()	RecvWorkerThread와 SendWorkerThread 생성
5	요청 생성	RequestXxx()	Protobuf C_* 메시지 작성
6	프레이밍	MakeSendBuffer()	[size:uint16][id:uint16][payload] 생성
7	비동기 송신	SendPacketQueue → SendWorker	부분 송신을 반복해 전체 바이트 전송
8	비동기 수신	RecvWorker	헤더 4바이트 후 payload 정확 길이 수신
9	게임 스레드 전달	RecvPacketQueue	완성된 한 패킷을 큐에 적재
10	디스패치	HandleRecvPackets()	패킷 ID로 핸들러 선택, Protobuf 파싱
11	상태/UI 반영	HandleRoomXxx + Delegate	TArray/USTRUCT 갱신 후 Blueprint Broadcast
12	종료	DisconnectFromGameServer()	퇴장 패킷 enqueue 후 소켓 파괴; 순서 보장 필요

송신 경로

주요 단계	호출 데이터	비고
Blueprint/UI	RequestCreateRoom(name, max)	입력 trim/검증
GameInstance	Protocol::C_CREATE_ROOM 작성	UTF-8 변환, 필드 설정
매크로	SEND_PACKET(Pkt)	MakeSendBuffer 후 GameInstance::SendPacket
패킷 핸들러	ByteSizeLong + PacketHeader + SerializeToArray	ID 1004, 전체 크기 기록
PacketSession	SendPacketQueue.Enqueue	게임 → 송신 스레드 전달
SendWorker	Dequeue → SendDesiredBytes	부분 송신을 고려해 포인터/잔여 길이 갱신
OS/TCP	Socket->Send	서버 바이트 스트림으로 전달

수신 경로

계층	호출/메시지	특징
OS/TCP	바이트 도착	메시지 경계가 없으므로 길이 기반 조립 필요
RecvWorker	헤더 4바이트 수신	PacketSize와 PacketID 역직렬화
RecvWorker	PacketSize - 4만큼 payload 수신	한 패킷 TArray 완성
PacketSession	RecvPacketQueue.Enqueue	네트워크 → 게임 스레드 전달
Game loop/BP	HandleRecvPackets() 호출	큐가 빌 때까지 drain
ServerPacketHandler	GPacketHandler[id]	템플릿이 Protobuf ParseFromArray 수행
콘텐츠 핸들러	Handle_S_ROOM_LIST 등	GameInstance 메서드 호출
GameInstance/UI	상태 갱신 + Broadcast	Blueprint 위젯이 변경을 반영

4. 기능별 시나리오 흐름

기능	호출	흐름	상태
연결	ConnectToGameServer	TCP Connect → PacketSession 생성 → 2개 워커 시작	구현
로그인	C_LOGIN(1000)	S_LOGIN(1001) success → Lobby 레벨 OpenLevel	구현
방 목록	C_ROOM_LIST(1002)	S_ROOM_LIST → LobbyRooms 재구성 → OnRoomListUpdated	구현
방 생성	C_CREATE_ROOM(1004)	이름/정원 검증 → S_CREATE_ROOM → CurrentRoom → 결과 Delegate	구현
방 입장	C_ENTER_ROOM(1006)	RoomId 검증 → S_ENTER_ROOM → CurrentRoom → 결과 Delegate	구현
팀 변경	C_CHANGE_TEAM(1011)	UE enum을 Protocol enum으로 변환; S_ROOM_STATE에서 최종 반영	구현
방 상태	S_ROOM_STATE(1012)	RoomInfo를 CurrentRoom으로 투영 → OnRoomStateUpdated	구현
방 퇴장	C/S_LEAVE_ROOM	Disconnect 시 enqueue는 하나 RequestLeaveRoom은 항상 false	미완성
매치	1013~1019, 1023~1024	ID와 파서만 있으며 콘텐츠 핸들러는 대부분 true 반환	골격
이동 송신	C_MOVE(1020)	입력 변경 즉시 또는 0.2초마다 MoveInfo 전송	구현
이동 수신	S_MOVE(1021)	핸들러 연결은 있으나 GameInstance 적용부가 주석	미완성
디스폰/채팅	1022, 1025~1026	패킷 정의는 있으나 실제 콘텐츠 반영 미구현	골격

이동 동기화 상세

기능	호출/메시지	호출/메시지	호출/메시지
입력	Move()가 카메라 yaw 기준 방향 계산, AddMovementInput	없음	없음
매 Tick	부모 Tick이 실제 위치를 PlayerInfo에 기록	입력 변화 감지 및 상태 IDLE/MOVE 설정	부모 Tick에서 상태 기반 이동 예정
즉시/0.2초	C_MOVE에 현재 MoveInfo와 DesiredYaw 복사	송신 큐 → 서버	없음
서버 중계	없음	S_MOVE(PlayerInfo) 수신 예정	클라이언트 핸들러 진입
적용	자기 패킷이면 무시해야 함	게임 스레드에서 액터 탐색	SetDestInfo 후 보간/예측 필요; 현재 주석 처리

5. 패킷 ID와 구현 상태

ID	메시지	방향	종료/이전후 상태
1000	C_LOGIN	C→S	송신 구현
1001	S_LOGIN	S→C	로그인/레벨 전환
1002	C_ROOM_LIST	C→S	송신 구현
1003	S_ROOM_LIST	S→C	목록/Delegate
1004	C_CREATE_ROOM	C→S	송신 구현
1005	S_CREATE_ROOM	S→C	방 상태/Delegate
1006	C_ENTER_ROOM	C→S	송신 구현
1007	S_ENTER_ROOM	S→C	방 상태/Delegate
1008	C_LEAVE_ROOM	C→S	Disconnect에서만 enqueue
1009	S_LEAVE_ROOM	S→C	빈 핸들러
1010	C_CHANGE_PLAYER_TYPE	C→S	버퍼 생성 지원, 호출 없음
1011	C_CHANGE_TEAM	C→S	송신 구현
1012	S_ROOM_STATE	S→C	방 상태/Delegate
1013	C_START_MATCH	C→S	버퍼 생성 지원, 호출 없음
1014	S_MATCH_PREPARE	S→C	빈 핸들러
1015	C_MATCH_READY	C→S	버퍼 생성 지원, 호출 없음
1016	S_MATCH_START	S→C	빈 핸들러
1017	S_MATCH_STATE	S→C	빈 핸들러
1018	S_PLAYER_STATE	S→C	빈 핸들러
1019	S_PLAYER_RESPAWN	S→C	빈 핸들러
1020	C_MOVE	C→S	0.2초/변경 시 송신
1021	S_MOVE	S→C	호출되나 적용부 주석
1022	S_PLAYER_DESPAWN	S→C	빈 핸들러
1023	S_MATCH_END	S→C	빈 핸들러
1024	S_RETURN_TO_ROOM	S→C	빈 핸들러
1025	C_CHAT	C→S	버퍼 생성 지원, 호출 없음
1026	S_CHAT	S→C	빈 핸들러

패킷 테이블은 65,535개의 std::function 슬롯을 전역 배열로 확보하고 모든 슬롯을 INVALID 핸들러로 채운 뒤, 서버 패킷 ID만 램다로 덮어쓴다. 조회는 O(1)이지만 정적 초기화 비용과 메모리 사용, 마지막 uint16 값(65535) 처리에는 주의가 필요하다.

6. 서버 데이터가 UI 상태로 바뀌는 과정

Protocol 관련	종료/이전후 관련	변경 관련	UI 관련
S_ROOM_LIST.rooms	TArray LobbyRooms	Reset 후 room_id/name/count/max/host 복사	OnRoomListUpdated

Protobuf 이름	클라이언트 호스트	호스트 구조	비고
RoomInfo	FCurrentRoomState	기본 필드 복사, players를 FRoomPlayerItem 배열로 변환	호출자가 결과/상태 Delegate
Protocol::Team	ERoomTeam	TEAM_RED/BLUE, 나머지는 None	OnRoomStateUpdated
S_CREATE_ROOM	CurrentRoom	success && has_room_info일 때만 갱신	OnCreateRoomResult(bool)
S_ENTER_ROOM	CurrentRoom	success && has_room_info일 때만 갱신	OnEnterRoomResult(bool)
MoveInfo	AS1Player::PlayerInfo/DestInfo	현재 값/목적 값 분리	현재 별도 Delegate 없음

장점은 Protobuf 타입을 블루프린트에 직접 노출하지 않고 UE 리플렉션 타입으로 투영한다는 점이다. 반면 현재 FRoomPlayerItem에는 player_type과 ready 상태가 없고, ERoomReady는 선언만 되어 있어 UI가 필요한 모든 서버 상태를 아직 표현하지 못한다.

7. 코드 리뷰 관점의 핵심 위험과 개선 우선순위

우선순위	문제	위험	개선
P0	RequestReady 선언만 있고 정의 없음	UFUNCTION 링크/호출 실패 가능	C_MATCH_READY 의미와 인자를 확정해 구현하고 빌드 검증
P0	수신 헤더/길이 검증 부족	PacketSize < 4, 과대 길이, 잘못된 ID로 OOB/대용량 할당/파싱 오류	최소-최대 크기, ID 범위, payload 일치 검증 후 연결 종료
P0	워커 종료를 기다리지 않고 소켓 파괴	다른 스레드가 파괴된 FSocket에 접근 가능	Stop 플래그 → Shutdown으로 블로킹 해제 → WaitForCompletion → 소켓 Destroy
P1	Running이 일반 bool	스레드 간 data race	TAtomic 또는 UE thread-safe stop primitive
P1	송수신 루프 busy-wait	데이터가 없을 때 CPU 코어 지속 점유	event/select/wait 또는 짧은 backoff, blocking I/O 정책 명시
P1	퇴장 enqueue 직후 소켓 파괴	C_LEAVE_ROOM이 실제 전송되기 전에 유실	flush/ack/timeout을 가진 정상 종료 상태 머신
P1	GWorld 및 Cast 결과 무검증 매크로	월드 전환/종료 시 null 또는 잘못된 GameInstance	WorldContext/weak reference 사용, 명시적 함수와 실패 반환
P1	S_MOVE 핸들러에서 GWorld null 검사 없음	레벨 전환 타이밍 크래시 가능	다른 핸들러와 동일한 방어 코드 또는 중앙 컨텍스트
P1	uint16 크기 캐스팅 후 overflow	큰 Protobuf가 잘린 길이로 직렬화	ByteSizeLong이 UINT16_MAX-header 이하인지 선검증
P1	호스트 endian/구조체 레이아웃 의존	이기종 플랫폼/패딩 변화 시 프로토콜 불일치	명시적 4바이트 wire header, network byte order, static_assert
P2	HandleRecvPackets 외부 호출 의존	Tick 연결이 빠지면 큐만 쌓이고 UI가 멈춤	GameInstanceSubsystem/Tickable 객체로 수신 drain 책임 고정
P2	수신 큐 무제한 drain	패킷 폭주 시 한 프레임 hitch	프레임당 개수/시간 budget과 backlog 계측
P2	빈 핸들러가 true 반환	기능 미구현이 정상 처리처럼 보임	NotImplemented 로그/통계/명시적 상태
P2	raw new/delete MoveInfo	불필요한 heap과 소유권 복잡도	MoveInfo 값 멤버 또는 unique ownership
P2	Players raw UObject pointer	파괴된 액터를 가리킬 가능성	TWeakObjectPtr, UPROPERTY, Remove 시점 일원화

문제	문제	문제	문제
P2	원격 이동 보간 미완성	끊김-순간이동-지연 체감	snapshot buffer + interpolation, teleport threshold, timestamp/sequence
P3	BufferReader/Writer 연산자 경계 검사 없음	재사용 시 메모리 범위 초과	checked API만 노출하거나 ensure/optional 반환
P3	패킷 ID/핸들러 수동 중복	스키마 변경 시 클라이언트-서버 drift	generator를 단일 소스로 사용하고 CI diff 검증

8. 면접에서 설명할 때의 답변 프레임

30초 요약

“Unreal Engine 클라이언트에서 TCP와 Protobuf를 이용한 로비/방/이동 네트워크 계층을 구현했습니다. 송수신은 FRunnable 워커로 분리하고 TQueue로 게임 스레드에 전달해 UObject 접근을 게임 스레드에 한정했습니다. 패킷 ID 디스패치와 Blueprint Delegate를 통해 네트워크 응답이 UI 상태로 이어지도록 구성했으며, 현재는 로비 흐름이 완성되고 인게임 동기화는 확장 중입니다.”

2분 구조 설명

문제	문제	문제
문제	TCP는 메시지 경계가 없고 네트워크 스레드에서 UObject를 만지면 위험하다.	ReceiveDesiredBytes, 큐 경계
선택	4바이트 길이/ID 헤더와 Protobuf payload, 송수신 워커 2개를 선택했다.	PacketHeader, MakeSendBuffer, Run
흐름	요청은 송신 큐, 응답은 수신 큐를 거쳐 게임 스레드에서 ID 기반 처리한다.	PacketSession, GPacketHandler
UI	GameInstance가 서버 DTO를 UE USTRUCT로 바꾸고 Delegate로 위젯을 갱신한다.	UpdateCurrentRoom, Broadcast
트레이드 오프	단순성과 개발 속도는 좋지만 busy-wait, 종료 동기화, 검증 부족을 개선해야 한다.	NetworkWorker, Disconnect
다음 단계	정상 종료 상태 머신, 패킷 검증, 수신 budget, 원격 이동 snapshot 보간을 추가한다.	개선 계획

좋은 답변의 형태

좋은 답변	좋은 답변
“스레드를 써서 빠르게 했습니다.”	“소켓 I/O가 게임 스레드를 막지 않게 분리했고, UObject 변경은 수신 큐를 drain하는 게임 스레드로 제한했습니다.”
“TCP라서 안정적입니다.”	“TCP는 순서-재전송을 제공하지만 메시지 경계는 없으므로 size header와 exact-read 루프가 필요합니다.”
“Protobuf가 편해서 썼습니다.”	“스키마 기반 생성 코드와 하위 호환 필드 규칙을 얻는 대신 framing, 버전 배포, 최대 크기 검증은 별도로 설계했습니다.”
“아직 구현 안 됐습니다.”	“로비 vertical slice를 먼저 완성했고, 인게임은 핸들러 골격까지 연결했습니다. 다음 우선순위는 수명 안전성과 이동 보간입니다.”

9. 기술면접 질문 은행 - 총 205문항

각 문항 아래의 '답변 키워드'는 외워 읽는 정답이 아니라, 현재 코드의 구체적인 근거를 빠뜨리지 않기 위한 체크리스트다. 실전에서는 결론 → 코드 근거 → 트레이드오프 → 개선 순서로 40~90초 안에 답한다.

A. 프로젝트 개요와 설계 의도

#	질문과 답변 키워드
1	Q. 이 프로젝트를 한 문장으로 설명해 보세요. 답변 키워드: UE5 클라이언트, TCP/Protobuf, 로비·방·이동 vertical slice.
2	Q. 본인이 직접 구현한 범위와 템플릿/생성 코드의 경계는 무엇인가요? 답변 키워드: S1/Network-GameInstance-핸들러-플레이어 vs UE 템플릿*.pb*.
3	Q. 가장 어려웠던 기술적 문제는 무엇이었나요? 답변 키워드: 스레드 경계, TCP framing, UI 상태 연결 중 하나를 증거와 함께.
4	Q. 왜 GameInstance를 중심 객체로 선택했나요? 답변 키워드: 레벨 전환 생존, Blueprint 접근성, 전역 세션/로비 상태; 비대화 트레이드오프.
5	Q. 현재 완성도는 어디까지라고 평가하나요? 답변 키워드: 로그인/로비/방은 구현, 매치·채팅·원격 이동·정상 퇴장은 골격/미완성.
6	Q. 프로젝트의 핵심 데이터 흐름을 30초 안에 설명해 보세요. 답변 키워드: UI→C_*→프레이밍→송신 큐→TCP→수신 큐→ID dispatch→상태→Delegate.
7	Q. 이 구조에서 가장 잘한 설계 선택은 무엇인가요? 답변 키워드: 네트워크 스레드와 게임 스레드 분리, weak reference로 순환 방지.
8	Q. 지금 다시 만든다면 가장 먼저 바꿀 부분은 무엇인가요? 답변 키워드: 종료 상태 머신·검증·이벤트 기반 대기 우선.
9	Q. 기능을 어떤 순서로 구현했으며 그 이유는 무엇인가요? 답변 키워드: 연결→로그인→로비→방→이동의 vertical slice와 검증 가능성.
10	Q. 클라이언트가 권위(authority)를 가지는 데이터는 무엇인가요? 답변 키워드: 입력은 클라이언트, 최종 방/매치/위치 판정은 서버 권위가 바람직.
11	Q. 이 프로젝트에서 관찰 가능성(observability)은 어떻게 확보했나요? 답변 키워드: 현재 화면 메시지/UE_LOG, 향후 packet counters-latency-queue depth.
12	Q. 포트폴리오에서 이 프로젝트로 무엇을 증명하려 하나요? 답변 키워드: UE 런타임-C++·네트워크·동시성·프로토콜·UI 연결 능력.

B. Unreal Engine 구조와 생명주기

#	질문과 답변 키워드
13	Q. UGameInstance의 생명주기와 장단점은 무엇인가요? 답변 키워드: 프로세스 게임 세션 동안 유지, 레벨 전환 생존; 책임 집중과 테스트 난이도.
14	Q. GameInstance 대신 GameInstanceSubsystem을 쓰면 무엇이 좋아지나요? 답변 키워드: 책임 분리, 자동 생명주기, 모듈성·테스트성.
15	Q. GameMode와 GameInstance의 차이는 무엇인가요? 답변 키워드: GameMode는 월드/서버 권위, GameInstance는 레벨 간 지속 로컬 객체.
16	Q. GWorld 사용이 왜 위험할 수 있나요? 답변 키워드: 멀티월드/PIE, 레벨 전환, 종료 시 null, 테스트 컨텍스트 불명확.
17	Q. OpenLevel 직전/직후 네트워크 세션은 어떻게 유지되나요? 답변 키워드: GameInstance는 유지되지만 월드 참조와 핸들러 타이밍을 방어해야 함.
18	Q. UFUNCTION(BlueprintCallable)의 비용과 역할은 무엇인가요? 답변 키워드: 리플렉션 래퍼, Blueprint API; 고빈도 hot path 남용 주의.
19	Q. Dynamic Multicast Delegate를 선택한 이유는 무엇인가요? 답변 키워드: 여러 UI 구독자, Blueprint 바인딩; 해제·중복 바인딩 주의.
20	Q. UPROPERTY가 없는 UObject 포인터의 위험은 무엇인가요? 답변 키워드: GC 추적 부재와 dangling pointer; TWeakObjectPtr/UPROPERTY 고려.
21	Q. ACharacter Tick에서 네트워크 패킷을 만드는 장단점은 무엇인가요? 답변 키워드: 상태 접근 간단; 프레임 의존·모든 Pawn Tick 비용·연결 상태 낭비.
22	Q. Enhanced Input MappingContext는 언제 추가되나요? 답변 키워드: Controller 변경 시 LocalPlayerSubsystem에 우선순위 0으로 추가.
23	Q. MoveAction의 Completed도 Move에 바인딩한 이유는 무엇인가요? 답변 키워드: 0 입력을 캐시해 즉시 정지 패킷을 보내기 위함.
24	Q. bRunPhysicsWithNoController는 원격 캐릭터에 왜 필요한가요? 답변 키워드: 컨트롤러 없는 proxy도 CharacterMovement가 동작하도록.
25	Q. 레벨 전환 중 수신 패킷을 안전하게 처리하려면? 답변 키워드: 상태 머신, 월드 유효성, 보류 큐/드롭 정책, travel 완료 후 재개.
26	Q. Blueprint와 C++의 책임을 어떻게 나누겠습니까? 답변 키워드: C++ 네트워크·검증·상태, Blueprint 화면 흐름·표현; API는 좁게.
27	Q. Dedicated server와 이 클라이언트 코드의 역할 차이는? 답변 키워드: 서버 권위 시뮬레이션/검증, 클라이언트 입력·표현·예측.

C. C++ 메모리와 객체 수명

#	질문과 답변 키워드
28	Q. TSharedPtr와 TWeakPtr를 함께 쓴 이유는 무엇인가요? 답변 키워드: 세션 소유 유지와 Worker→Session 순환 참조 방지.
29	Q. AsShared()를 생성자에서 호출하면 왜 위험한가요? 답변 키워드: shared ownership이 아직 성립 전; 현재는 Run에서 호출.
30	Q. FSocket raw pointer는 누가 소유하나요? 답변 키워드: SocketSubsystem 생성/파괴; 명시적 단일 소유와 종료 순서 필요.
31	Q. MoveInfo를 raw new/delete 대신 값으로 두면 어떤 이점이 있나요? 답변 키워드: heap 제거, 예외/복사 안전성, 소유권 단순화.
32	Q. AS1Player 소멸자에서 delete하는 방식의 잠재 문제는? 답변 키워드: 향후 복사/이동 또는 중복 수명 관리; UObject는 비복사지만 값 멤버가 더 단순.
33	Q. SendBuffer가 TSharedFromThis인 이유는 무엇인가요? 답변 키워드: 큐와 송신 워커 사이에서 버퍼 수명을 공유.
34	Q. CopyData에서 len 검증이 없으면 어떤 문제가 생기나요? 답변 키워드: capacity 초과 memcpy; precondition/assert 또는 bounds check 필요.
35	Q. reinterpret_cast로 헤더를 쓰는 방식의 위험은? 답변 키워드: alignment, padding, endian, strict aliasing/플랫폼 호환성.
36	Q. TArray::GetData 포인터의 유효 기간은 언제까지인가요? 답변 키워드: 재할당/파괴 전까지; Add/SetNum 뒤 기존 포인터 무효 가능.
37	Q. TMap에서 액터 파괴 후 해야 할 일은? 답변 키워드: Remove, weak pointer/validity 검사, MyPlayer 정리.
38	Q. PacketSession 소멸자에서 Disconnect 호출의 재진입 문제는 없나요? 답변 키워드: 명시적 Disconnect와 소멸자 중복 호출을 idempotent하게 설계.
39	Q. RAII로 네트워크 자원을 관리한다면 어떻게 설계하나요? 답변 키워드: Socket deleter wrapper, thread join wrapper, session state enum.
40	Q. uint16 캐스팅 전에 범위를 확인해야 하는 이유는? 답변 키워드: narrowing 후 overflow를 감지할 수 없기 때문.
41	Q. std::function 65,535개 전역 배열의 비용은? 답변 키워드: 정적 메모리/초기화 비용; sparse map이나 작은 dense range 비교.
42	Q. BufferedReader::operator>>에 경계 검사가 없는 이유와 개선책은? 답변 키워드: 편의 API가 위험; bool/expected 반환, checked cursor.

D. TCP와 패킷 프레이밍

#	질문과 답변 키워드
43	Q. TCP에 패킷 경계가 없다는 말은 무엇인가요? 답변 키워드: 한 번 Send가 한 번 Recv와 대응하지 않음; stream에서 길이만큼 조립.
44	Q. 왜 헤더에 size와 id를 넣었나요? 답변 키워드: frame 경계 결정과 메시지 타입 dispatch.
45	Q. 부분 송신과 부분 수신은 왜 발생하나요? 답변 키워드: 커널 버퍼-네트워크 상황 때문에 요청 길이보다 적게 처리 가능.
46	Q. SendDesiredBytes가 부분 송신을 어떻게 처리하나요? 답변 키워드: BytesSent만큼 포인터 이동, 남은 Size 반복.
47	Q. RecvWorker는 sticky packet/coalescing을 처리할 수 있나요? 답변 키워드: 정확 길이 read를 반복하므로 기본 처리 가능; 선검증 필요.
48	Q. PacketSize가 4보다 작으면 현재 어떤 일이 생기나요? 답변 키워드: 음수 payload가 int32로 생기고 잘못된 AddZeroed/로직; 즉시 거부해야 함.
49	Q. 최대 패킷 크기를 왜 제한해야 하나요? 답변 키워드: 메모리 DoS와 uint16 overflow 방지.
50	Q. 네트워크 byte order를 쓰지 않으면 어떤 문제가 있나요? 답변 키워드: 다른 endian 아키텍처와 비호환.
51	Q. TCP keepalive와 애플리케이션 heartbeat의 차이는? 답변 키워드: OS 연결 감지 vs 게임 세션/지연/상태 확인.
52	Q. 서버가 연결을 정상 종료했을 때 Recv는 무엇을 반환하나요? 답변 키워드: 0바이트; 루프를 종료하고 disconnect 이벤트로 전파해야 함.
53	Q. Socket::HasPendingData를 busy loop에서 호출하는 문제는? 답변 키워드: 데이터 없을 때 CPU 소모와 종료 지연 정책 불명확.
54	Q. blocking과 non-blocking 소켓 중 무엇을 선택하겠습니까? 답변 키워드: 워커 전용이면 blocking+shutdown/join 단순, non-blocking이면 wait/select 필요.
55	Q. Nagle 알고리즘이 이동 패킷에 미치는 영향은? 답변 키워드: 작은 패킷 지연 가능; TCP_NODELAY를 측정 후 결정.
56	Q. 패킷 손실은 TCP가 해결하는데도 게임 지연이 생기는 이유는? 답변 키워드: 재전송과 head-of-line blocking.
57	Q. 이동에 UDP를 고려할 기준은 무엇인가요? 답변 키워드: 최신 상태 우선, 손실 허용, 자체 순서/보안/혼잡 제어 비용.
58	Q. 연결 재시도는 어떤 backoff가 적절한가요? 답변 키워드: 지수 backoff+jitter, 최대 횟수, 사용자 취소.
59	Q. graceful shutdown 순서를 설명해 보세요. 답변 키워드: 새 송신 중단→leave/flush/ack 또는 timeout→socket shutdown→thread join→destroy.
60	Q. 프로토콜에 sequence number가 필요한 이유는? 답변 키워드: 중복·오래된 상태 판별, RTT/손실 계측, UDP 전환 대비.
61	Q. TLS를 적용한다면 어느 계층에 넣겠습니까? 답변 키워드: transport wrapper, 인증/키 검증, payload framing은 암호화 채널 내부.
62	Q. 패킷 flood를 클라이언트에서 어떻게 방어하나요? 답변 키워드: size/rate/queue limit, unknown ID 차단, disconnect policy.

E. 멀티스레딩과 큐

#	질문과 답변 키워드
63	Q. 왜 송신과 수신 스레드를 분리했나요? 답변 키워드: 독립 blocking 가능성과 구현 단순성; 스레드 2개 비용.
64	Q. 네트워크 스레드에서 UObject를 직접 만지면 왜 위험한가요? 답변 키워드: 대부분 game-thread affinity, GC/월드 변경과 경쟁.
65	Q. TQueue가 스레드 경계를 어떻게 단순화하나요? 답변 키워드: 소유권 전달과 처리 컨텍스트 분리.
66	Q. 현재 TQueue의 생산자/소비자 모델은 무엇인가요? 답변 키워드: Recv: worker→game, Send: game→worker; 사실상 SPSC.
67	Q. Running bool은 왜 data race인가요? 답변 키워드: 서로 다른 스레드의 동기화 없는 읽기/쓰기.
68	Q. Destroy에서 WaitForCompletion이 필요한 이유는? 답변 키워드: 객체/소켓 파괴 전에 Run 종료를 보장.
69	Q. 스레드가 Recv에서 막혀 있으면 어떻게 깨우나요? 답변 키워드: socket shutdown/close 또는 cancel/event 후 join.
70	Q. busy-wait를 없애는 방법은? 답변 키워드: blocking queue/event, socket wait/select, condition variable.
71	Q. 수신 큐가 무한히 커질 때 어떤 현상이 생기나요? 답변 키워드: 메모리 증가, 지연 누적, stale state 처리.
72	Q. 한 프레임에 큐를 전부 비우는 것의 단점은? 답변 키워드: packet burst가 game thread hitch 유발.
73	Q. 프레임 budget 기반 drain은 어떻게 구현하나요? 답변 키워드: 최대 N개 또는 시간 budget, backlog metric, 중요도별 처리.
74	Q. 메모리 가시성은 어떤 원리로 확보되나요? 답변 키워드: thread-safe queue의 synchronization; 별도 공유 상태는 atomic/lock 필요.
75	Q. TWeakPtr::Pin 실패 시 어떤 의미인가요? 답변 키워드: 세션 수명 종료; worker는 I/O와 루프를 종료해야 함.
76	Q. 세션 재연결 중 이전 워커와 새 워커가 겹치면? 답변 키워드: 중복 송수신/오래된 큐; generation/session ID와 확실한 join 필요.
77	Q. 멀티프로듀서가 SendPacketQueue에 접근할 가능성은? 답변 키워드: 게임 스레드 외 호출을 금지하거나 MPSC 모드/문서화.
78	Q. lock-free가 항상 빠른가요? 답변 키워드: 경합·캐시·allocation-backpressure 비용을 측정해야 함.
79	Q. 패킷 순서는 큐와 TCP에서 보장되나요? 답변 키워드: 각 연결 TCP 및 FIFO 큐는 순서 보장; 여러 생산자/재연결은 별도.
80	Q. 게임 종료 시 스레드 정리 테스트는 어떻게 하나요? 답변 키워드: 반복 connect/disconnect, travel, 강제 서버 종료, sanitizer/log/thread count.
81	Q. 워커 예외/실패를 게임 스레드에 어떻게 알리나요? 답변 키워드: connection event queue와 상태 enum/delegate.
82	Q. thread-per-connection과 IOCP의 차이는? 답변 키워드: 단순성 vs 대규모 연결 확장성; 클라이언트 한 연결은 전자가 가능.

F. Protobuf와 버전 관리

#	질문과 답변 키워드
83	Q. Protobuf를 선택한 이유는 무엇인가요? 답변 키워드: 스키마·생성 코드·compact binary·다언어 지원.
84	Q. Protobuf가 패킷 framing까지 제공하나요? 답변 키워드: 아니며 TCP 길이/타입 framing은 애플리케이션 책임.
85	Q. ParseFromArray 실패는 언제 발생하나요? 답변 키워드: 잘린 payload, 잘못된 wire type/length, required 규칙(버전별).
86	Q. 필드 번호를 재사용하면 왜 안 되나요? 답변 키워드: 구버전 데이터가 새 의미로 해석될 수 있음; reserved 처리.
87	Q. 필드 추가가 대체로 호환되는 이유는? 답변 키워드: unknown field를 건너뛰고 기본값 사용.
88	Q. enum 확장 시 클라이언트는 어떻게 대응해야 하나요? 답변 키워드: unknown 값을 안전한 None/Unknown으로 매핑하고 로그.
89	Q. has_room_info를 검사하는 이유는? 답변 키워드: 메시지 필드 존재 여부와 실패 응답을 구분.
90	Q. ByteSizeLong과 SerializeToArray 사이 데이터가 바뀌면? 답변 키워드: 크기 불일치 가능; 동일 스레드 로컬 객체로 유지.
91	Q. 생성된 pb.cc를 직접 수정하면 안 되는 이유는? 답변 키워드: 재생성 시 유실, 스키마와 drift.
92	Q. 클라이언트와 서버 스키마 버전을 어떻게 동기화하나요? 답변 키워드: 단일 proto 원본, generator, CI에서 생성물 diff/호환성 검사.
93	Q. 패킷 ID와 메시지 타입 매핑도 생성해야 하나요? 답변 키워드: 수동 중복 제거와 불일치 방지에 유리.
94	Q. 문자열을 TCHAR_TO_UTF8/UTF8_TO_TCHAR로 바꾸는 이유는? 답변 키워드: UE FString 표현과 protobuf UTF-8 string 변환.
95	Q. 악성 Protobuf payload 방어는 어떻게 하나요? 답변 키워드: frame size, recursion/total bytes limit, parse 실패 disconnect/rate limit.
96	Q. JSON 대신 Protobuf의 단점은? 답변 키워드: 가독성·수동 디버깅 저하, 스키마/생성 파이프라인 필요.
97	Q. 메시지 압축은 언제 고려하나요? 답변 키워드: 큰 반복 데이터에서 측정 후; 작은 실시간 패킷은 헤더/CPU 비용이 더 큼.

G. 로비·방·UI 상태 관리

#	질문과 답변 키워드
98	Q. RoomInfo를 UE USTRUCT로 변환하는 이유는? 답변 키워드: Blueprint/UPROPERTY 노출과 protobuf 의존 격리.
99	Q. LobbyRooms를 Reset 후 재구성하는 장단점은? 답변 키워드: 단순·일관; 큰 목록에서 allocation/UI 전체 갱신 비용.
100	Q. 증분 업데이트가 필요해지는 기준은? 답변 키워드: 방 수·업데이트 빈도·UI diff 비용 증가.
101	Q. 서버 응답 success와 has_room_info를 둘 다 보는 이유는? 답변 키워드: 논리 성공과 payload 존재의 독립 검증.
102	Q. 방 생성/입장 결과 Delegate에 bool만 전달하는 한계는? 답변 키워드: 실패 사유/재시도/표시 문구 없음; error enum/message 필요.
103	Q. 팀 변경 응답이 별도 S_CHANGE_TEAM이 아닌 S_ROOM_STATE인 장점은? 답변 키워드: 최종 authoritative snapshot으로 모든 참가자 일관.
104	Q. CurrentPlayerCount를 players_size로 계산하는 장점은? 답변 키워드: 중복 필드 불일치 방지.
105	Q. ERoomReady가 미사용인 점을 어떻게 설명하나요? 답변 키워드: 기능 확장 흔적; 구현 전 제거하거나 프로토콜 상태와 연결.
106	Q. 호스트가 나가면 HostObjectId는 어떻게 갱신돼야 하나요? 답변 키워드: 서버가 새 host를 결정하고 room snapshot broadcast.
107	Q. 중복 방 입장 응답이나 늦은 응답은 어떻게 막나요? 답변 키워드: request ID/state enum/generation으로 현재 요청과 매칭.
108	Q. UI가 Delegate를 언제 bind/unbind해야 하나요? 답변 키워드: Construct/Destruct 또는 활성 생명주기, 중복 바인딩 방지.
109	Q. 네트워크 끊김 시 로비 UI 상태는 어떻게 표시하나요? 답변 키워드: connection state, 입력 비활성, 재연결/오류 reason.
110	Q. 방 목록 이름에 대한 클라이언트 검증만으로 충분한가요? 답변 키워드: 아니며 서버가 길이·문자·중복·권한을 재검증.
111	Q. 서버 snapshot을 로컬 캐시에 적용할 때 원자성이 필요한가요? 답변 키워드: 한 게임 스레드에서 전체 구조 갱신 후 한 번 Broadcast.
112	Q. MVVM을 도입한다면 어디를 분리하나요? 답변 키워드: GamelInstance service, ViewModel observable state, Widget view.

H. 이동 동기화와 게임 네트워킹

#	질문과 답변 키워드
113	Q. 0.2초 주기는 초당 몇 패킷인가? 답변 키워드: 기본 5Hz, 입력 변경 시 즉시 추가.
114	Q. 입력 변경 시 즉시 보내는 이유는? 답변 키워드: 출발/정지 반응 지연을 최대 200ms 줄임.
115	Q. 현재 위치와 DesiredYaw를 같이 보내는 이유는? 답변 키워드: 서버/원격이 위치 snapshot과 의도 방향을 알 수 있음.
116	Q. 클라이언트 위치를 서버가 그대로 믿으면 어떤 문제가 있나요? 답변 키워드: speed hack/teleport; 입력·속도·충돌 검증 필요.
117	Q. 원격 캐릭터를 AddMovementInput만으로 재현하면 왜 drift가 생기나요? 답변 키워드: 프레임/충돌/지연 차이로 위치 오차 누적.
118	Q. snapshot interpolation을 설명해 보세요. 답변 키워드: 과거 시점 버퍼 두 snapshot 사이를 보간해 jitter 흡수.
119	Q. extrapolation은 언제 쓰며 위험은? 답변 키워드: 새 snapshot 지연 시 예측; 방향 전환에서 오차.
120	Q. teleport threshold가 필요한 이유는? 답변 키워드: 큰 오차를 장시간 보간하지 않고 즉시 교정.
121	Q. sequence/timestamp가 이동 패킷에 왜 필요한가요? 답변 키워드: 오래된 snapshot 폐기, 시간축 보간, RTT 추정.
122	Q. 자기 플레이어의 서버 echo를 무시해도 되나요? 답변 키워드: 권위 교정이 필요하므로 무조건 무시보다 reconciliation.
123	Q. client-side prediction과 reconciliation은 무엇인가? 답변 키워드: 즉시 로컬 적용 후 서버 확정 시 미처리 입력 재적용.
124	Q. CharacterMovement replication을 쓰지 않고 커스텀한 이유를 어떻게 설명하나요? 답변 키워드: 학습/서버 연동/프로토콜 통제; 내장 기능 재사용과 비교 필요.
125	Q. 회전은 yaw만 보내도 충분한가요? 답변 키워드: 평면 캐릭터면 가능; pitch/roll/aim은 별도.
126	Q. 정지 입력에서 DesiredYaw 계산이 애매한 이유는? 답변 키워드: zero direction look-at; 마지막 유효 yaw 유지가 안전.
127	Q. 패킷 주기를 동적으로 조절한다면 기준은? 답변 키워드: 이동/정지, 가속·회전 변화, RTT, bandwidth, 시야 중요도.
128	Q. 관심 영역(AOI)은 클라이언트에 어떤 영향을 주나요? 답변 키워드: 수신 엔티티 수와 snapshot 빈도 감소, 스폰/디스폰 경계 처리.
129	Q. 매치 상태와 플레이어 상태를 분리한 이유는? 답변 키워드: 공용 점수/시간과 개별 HP/킬의 갱신 빈도·구독 분리.
130	Q. 사망/리스폰 중 이동 패킷은 어떻게 처리하나요? 답변 키워드: 서버 상태 머신에서 거부, 클라이언트 입력 잠금과 sequence reset.

I. 오류 처리·보안·복구

#	질문과 답변 키워드
131	Q. 잘못된 packet ID가 오면 현재 어떻게 되나요? 답변 키워드: 대부분 INVALID false; 65535는 배열 경계 문제 가능.
132	Q. 핸들러가 false를 반환한 뒤 현재 후속 조치는 있나요? 답변 키워드: 없음; 로그·metric-disconnect 정책 필요.
133	Q. 로그인 실패와 네트워크 실패를 UI에서 어떻게 구분하나요? 답변 키워드: 도메인 error code와 connection error state 분리.
134	Q. 서버가 비정상적으로 큰 room list를 보내면? 답변 키워드: frame/repeated count limit, UI paging, rate limit.
135	Q. 채팅 문자열에서 어떤 검증이 필요한가요? 답변 키워드: UTF-8, 길이, 금칙어/권한, rate limit, 표시 시 escaping.
136	Q. 재연결 후 이전 방에 복귀하려면 무엇이 필요한가요? 답변 키워드: session token, 서버 state query, idempotent rejoin, generation.
137	Q. 패킷 재전송이 중복 요청을 만들 수 있나요? 답변 키워드: TCP 자체 중복은 숨기지만 앱 재시도는 가능; request ID/idempotency.
138	Q. 클라이언트가 room_id를 조작하면? 답변 키워드: 서버가 존재·정원·상태·권한 검증.
139	Q. 로그에 민감 정보를 남기지 않으려면? 답변 키워드: 토큰/개인정보 redaction, payload 전문 금지, 레벨별 정책.
140	Q. 치트 방지를 클라이언트만으로 할 수 있나요? 답변 키워드: 불가; 서버 권위 검증과 탐지, 클라이언트는 UX 보조.
141	Q. 연결 타임아웃은 어디에 구현하나요? 답변 키워드: 비동기 connect/worker state machine과 game-thread timer.
142	Q. 프로토콜 버전 불일치를 어떻게 감지하나요? 답변 키워드: handshake version/capabilities, 지원 범위, 명확한 종료 사유.
143	Q. 서버 장애 시 thundering herd를 어떻게 막나요? 답변 키워드: jittered exponential backoff와 재시도 제한.
144	Q. 큐 overflow 정책은 drop-oldest와 disconnect 중 무엇인가요? 답변 키워드: 메시지 의미별: snapshot은 오래된 것 drop, 신뢰 이벤트는 backpressure/disconnect.
145	Q. 패킷 무결성 검사는 TCP checksum으로 충분한가요? 답변 키워드: 전송 오류 검출과 악성 변조는 다름; TLS/authentication 필요.

J. 테스트·디버깅·성능

#	질문과 답변 키워드
146	Q. 이 코드를 단위 테스트하기 어려운 이유는? 답변 키워드: GWorld/FSocket/Blueprint 결합; 인터페이스 주입 부족.
147	Q. 패킷 serializer를 어떻게 테스트하나? 답변 키워드: known message→bytes→header/id/parse round trip, 경계 크기.
148	Q. TCP fragmentation 테스트는 어떻게 하나? 답변 키워드: mock socket이 1~N 바이트씩 반환하도록 구성.
149	Q. 악성 헤더 fuzz test에는 어떤 케이스가 있나요? 답변 키워드: 0/1/3/4/65535 size, unknown/65535 id, 잘린 payload.
150	Q. connect/disconnect race를 어떻게 재현하나? 답변 키워드: 반복 루프, 전송 중 종료, 서버 강제 종료, level travel.
151	Q. 게임 스레드 hitch를 무엇으로 측정하나? 답변 키워드: Unreal Insights, stat game, queue drain 시간/개수.
152	Q. 네트워크 성능 지표는 무엇을 수집하나? 답변 키워드: bytes/s, packets/s, RTT, reconnect, parse failures, queue depth.
153	Q. busy-wait CPU 사용을 어떻게 확인하나? 답변 키워드: Unreal Insights/OS profiler에서 worker CPU와 call stack.
154	Q. 이동 품질을 정량화하는 지표는? 답변 키워드: position error, correction count, jitter, interpolation delay.
155	Q. 패킷 핸들러를 통합 테스트하는 방법은? 답변 키워드: recorded byte frames를 queue에 넣고 GameInstance state/delegate 검증.
156	Q. Blueprint Delegate 테스트는 어떻게 하나? 답변 키워드: automation test용 UObject subscriber와 broadcast count/payload.
157	Q. 서버가 느릴 때 UI가 멈추지 않는지 어떻게 검증하나? 답변 키워드: latency/loss proxy, 비동기 상태와 timeout, frame time.
158	Q. 메모리 누수를 찾는 방법은? 답변 키워드: 반복 세션 후 thread/socket/object count, LLM, sanitizer/VS diagnostics.
159	Q. 패킷 생성 코드 변경을 CI에서 어떻게 검증하나? 답변 키워드: proto 재생성 후 git diff가 깨끗한지, server/client build matrix.
160	Q. 로그 레벨을 어떻게 나누나요? 답변 키워드: connection/error는 warning, packet trace는 verbose와 샘플링.
161	Q. 실서버 없이 개발하려면? 답변 키워드: mock server, deterministic packet playback, loopback integration test.
162	Q. 성능 최적화 전에 무엇을 측정해야 하나? 답변 키워드: 실제 packet rate/size, allocation, queue contention, frame budget.
163	Q. 대규모 방 목록에서 allocation을 줄이는 방법은? 답변 키워드: Reserve, object reuse, paging/diff update.

K. 확장성과 아키텍처

#	질문과 답변 키워드
164	Q. GameInstance가 비대해지면 어떻게 분리하나요? 답변 키워드: ConnectionService, LobbyService, MatchService, replicated models/subsystems.
165	Q. 패킷 핸들러와 콘텐츠 로직 결합을 줄이는 방법은? 답변 키워드: typed event bus/command, service interface, dependency injection.
166	Q. 패킷 우선순위가 필요하면 큐를 어떻게 바꾸나요? 답변 키워드: control/reliable state/snapshot 채널 분리와 budget.
167	Q. 여러 서버(로그인/로비/게임)에 연결한다면? 답변 키워드: connection manager와 endpoint별 session, handoff token.
168	Q. 샤딩이나 채널 이동에서 세션 전환은? 답변 키워드: 새 서버 인증→상태 handoff→준비 확인→기존 연결 종료.
169	Q. 패킷 테이블 배열과 unordered_map의 선택 기준은? 답변 키워드: ID 밀도, 메모리, 조회 성능, 초기화 비용.
170	Q. 코드 생성으로 핸들러를 만들 때 얻는 이점은? 답변 키워드: ID/타입 일치, 반복 제거, schema drift 감소.
171	Q. request-response correlation이 필요한 기능은? 답변 키워드: 동시 생성/입장/매치 요청, timeout과 늦은 응답 식별.
172	Q. 상태 snapshot과 event sourcing의 차이는? 답변 키워드: 최종 상태 단순 복구 vs 이벤트 감사/재생과 순서 복잡도.
173	Q. ECS/Mass를 도입할 시점은? 답변 키워드: 많은 엔티티와 데이터 지향 업데이트가 병목일 때 측정 후.
174	Q. 플랫폼 간 프로토콜 호환성을 어떻게 보장하나요? 답변 키워드: 고정 wire format, endian, 크기, automated cross-platform fixtures.
175	Q. 핫 리로드/레벨 전환에서 delegate 수명은? 답변 키워드: 바인딩 객체 유효성, 해제, 중복 등록 테스트.
176	Q. 서비스 상태 머신에는 어떤 상태가 필요한가요? 답변 키워드: Disconnected/Connecting/Connected/Authenticating/Lobby/Room/Match/Closing/Error.
177	Q. 재시도 가능한 오류와 불가능한 오류를 어떻게 나누나요? 답변 키워드: timeout/일시 서버 오류 vs 인증/버전/제재 오류.
178	Q. 백프레서를 어디에서 걸 수 있나요? 답변 키워드: request rate, queue capacity, worker dequeue, server flow control.
179	Q. 한 연결에 송수신 스레드 2개가 과한 경우는? 답변 키워드: 모바일/다중 연결; event loop/async I/O와 비교.
180	Q. 프로토콜 문서화를 자동화한다면? 답변 키워드: proto + packet ID metadata로 표/테스트/핸들러 생성.
181	Q. 향후 음성/대용량 에셋 전송을 같은 채널에 넣을까요? 답변 키워드: HOL blocking 방지를 위해 별도 서비스/채널/CDN.

L. 현재 코드 기반 압박 질문

#	질문과 답변 키워드
182	Q. RequestReady는 선언됐는데 구현은 어디 있나요? 답변 키워드: 없음을 인정하고 빌드/링크 영향, C_MATCH_READY 설계 후 구현 계획.
183	Q. RequestLeaveRoom이 항상 false인데 퇴장은 된다고 말할 수 있나요? 답변 키워드: 정상 퇴장 API는 미완성; Disconnect의 enqueue도 전달 보장 없음.
184	Q. Disconnect에서 leave 패킷이 실제 전송됐다고 보장할 수 있나요? 답변 키워드: 보장 불가; ack/flush/join 전 소켓 파괴 문제.
185	Q. RecvWorker::Destroy는 스레드를 정말 종료시키나요? 답변 키워드: 플래그만 변경하며 join/블로킹 해제가 없어 보장 부족.
186	Q. Running을 atomic으로만 바꾸면 종료 문제가 전부 해결되나요? 답변 키워드: 아님; blocked I/O wakeup, join, socket lifetime 순서도 필요.
187	Q. 왜 SendWorker 루프에 Sleep이 없나요? 답변 키워드: 현재 결함/프로토타입; event 기반 queue wakeup이 권장.
188	Q. GPacketHandler[UINT16_MAX]에 id 65535가 오면? 답변 키워드: 배열 유효 인덱스는 0~65534라 OOB; 65536 또는 명시적 검증.
189	Q. HandlePacket은 len이 헤더보다 작은 경우를 막나요? 답변 키워드: 막지 않음; header 접근 전 len>=4 검증 필요.
190	Q. PacketSize와 실제 TArray 길이가 다르면 어디서 검증하나요? 답변 키워드: 현재 명시적 검증 없음; recv 단계와 dispatch 단계 양쪽 방어.
191	Q. MovePkt를 연결 전에도 매 Tick 만들 수 있나요? 답변 키워드: 가능; GameInstance SendPacket에서 드롭하지만 allocation/직렬화 비용 발생.
192	Q. S_MOVE가 와도 원격 캐릭터가 움직이나요? 답변 키워드: 아니며 HandleMove 본문이 주석 처리됨.
193	Q. Handle_S_MOVE에서 GWorld가 null이면? 답변 키워드: 즉시 역참조 위험; 다른 핸들러와 불일치.
194	Q. HandleDespawn이 TMap에서 항목을 제거하나요? 답변 키워드: 액터만 Destroy하고 Remove하지 않아 stale pointer 위험.
195	Q. HandleSpawn이 현재 플레이어 맵을 채우나요? 답변 키워드: 전체 본문 주석 처리로 채우지 않음.
196	Q. FRoomPlayerItem이 ready/player type을 표현하나요? 답변 키워드: 아니며 현재 object id와 team만 보유.
197	Q. C_CHANGE_PLAYER_TYPE에 success 필드가 왜 있나요? 답변 키워드: 요청 메시지 의미상 어색한 스키마; 서버 응답으로 분리 검토.
198	Q. 헤더 구조체 크기가 항상 4바이트인가? 답변 키워드: 현재 필드상 흔히 4지만 static_assert와 명시 직렬화가 필요.
199	Q. ReceiveDesiredBytes는 HasPendingData가 false면 연결 종료인가? 답변 키워드: 아니며 단순히 지금 데이터 없음일 수 있음; 현재 루프가 즉시 재시도.
200	Q. 수신 큐가 쌓였는데 HandleRecvPackets가 호출되지 않으면? 답변 키워드: 상태/UI가 갱신되지 않고 메모리 backlog 증가.
201	Q. 빈 콘텐츠 핸들러가 true를 반환하는 것이 좋은가요? 답변 키워드: 기능 누락을 숨길 수 있어 NotImplemented telemetry가 낫다.
202	Q. ServerPacketHandler::Init를 세션마다 65,535번 채우는 비용은? 답변 키워드: 재연결마다 반복; once/static 초기화 또는 작은 테이블.
203	Q. SEND_PACKET 매크로의 단점은? 답변 키워드: GWorld/타입/실패 숨김, 디버깅과 테스트 어려움; 함수로 대체.
204	Q. DesiredInput과 DesiredYaw의 초기값은 명확한가요? 답변 키워드: 명시적 초기화를 권장하며 zero-input yaw 계산도 분리.
205	Q. 현재 코드를 shipping 수준이라고 평가하나요? 답변 키워드: 아니며 프로토타입/vertical slice; P0/P1 안정화와 테스트가 선행.

10. 7일 면접 대비 사용법

일차	주요 내용	면접 준비
1일차	1~4장 흐름을 보고 백지에 송신/수신 경로 재작성	30초·2분 프로젝트 소개 녹음
2일차	TCP framing, 부분 송수신, 종료 순서	D/L 문항 중 20개 구두 답변
3일차	스레드·큐·수명	E/C 문항과 graceful shutdown 의사코드
4일차	Protobuf·버전·보안	F/I 문항과 악성 패킷 검증표
5일차	UE 생명주기·Blueprint UI	B/G 문항과 GameInstance 분리 설계
6일차	이동 동기화·성능·테스트	H/J 문항과 snapshot 보관 그림
7일차	압박 질문 전체	L 문항을 '인정-영향-개선-검증' 형태로 모의면접

압박 질문 대응 공식

- ① 사실을 숨기지 않고 현재 상태를 정확히 인정한다. ② 실제 영향과 재현 조건을 말한다. ③ 수정 순서와 선택한 동기화/검증 방식을 제안한다. ④ 어떤 테스트와 지표로 완료를 증명할지 덧붙인다.

11. 분석 근거 파일

주요 모듈	주요 파일
부트/빌드	S1.uproject, Config/DefaultEngine.ini, Source/S1/S1.Build.cs, S1.cpp/h
세션	Source/S1/Network/PacketSession.h/.cpp
워커	Source/S1/Network/NetworkWorker.h/.cpp
프레이밍	Source/S1/ServerPacketHandler.h/.cpp, BufferReader/Writer
게임 상태	Source/S1/S1GameInstance.h/.cpp
플레이어	Source/S1/Game/S1Player.h/.cpp, S1MyPlayer.h/.cpp
스키마	Source/S1/Network/Enum.pb.*, Struct.pb.*, Protocol.pb.*
스키마 원본 참고	X:\Github\Server_std\Common\Protobuf\bin*.proto

주의: Blueprint 바이너리(.uasset)의 내부 노드 연결은 텍스트 정적 분석 범위 밖이다. 따라서 HandleRecvPackets의 실제 호출 주기와 위젯 Delegate 바인딩은 C++ 공개 API 및 설정을 근거로 설명했으며, 에디터 Blueprint 그래프에서 최종 확인해야 한다. 빌드/실행 테스트가 아니라 현재 소스 기준의 구조·위험 분석이다.